pMissionHash and pMissionEval

Mission hashing and mission-evaluation utilities.

Included MIT MOOS-IvP PDF sources:

- app_pmhash.pdf
- app_pmissioneval.pdf

pMissionHash: A Lightweight App for Generating a Mission Hash

January 28th 2025

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
project-pavlab/appdocs/app_pmhash

1 Overview

The pMissionHash app was meant to run and generate a mission hash when (a) pMarineViewer is *not* being run, and (b) mission hash generation is still desired. A mission hash is normally generated on the shoreside MOOS community in the form of a posting to the variable MISSION_HASH. Normally the hash is generated and posted by pMarineViewer. However, in headless missions, where there is no GUI and no pMarineViewer, a mission hash posting can be generated instead with pMissionHash. Headless mission configurations are common in automated missions, e.g., for validation, automated competitions, or Monte Carlo testing.

Why are mission hashes important? The mission hash is created and logged on the shoreside and shared to and logged on all the vehicles. The presence of identical mission hashes is the definitive confirmation that the individual log files are all part of the same instance of a particular mission, either in simulation or in the physical world. As of this writing, a mission hash is generated either with pMarineViewer or pMissionHash. The command-line tool mhash_gen is also distributed with MOOS-IvP but is not generally part of a mission.

2 Typical Application Topology

The pMissionHash app typically runs in the shoreside community, in a headless mission, when pMarineViewer is not running.

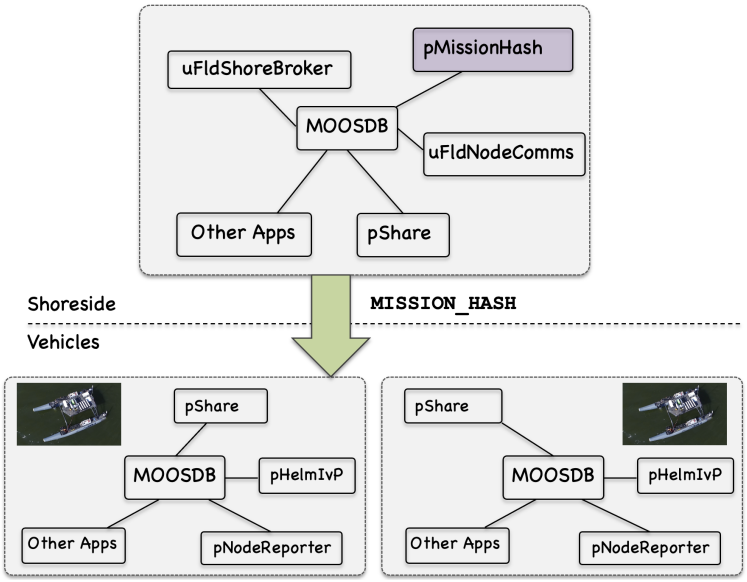

Figure 1: **Typical pMissionHash Topology:** A unique randomly created mission hash is generated by pMissionHash at startup, and shared to the MOOS communities of all vehicles. This unique has is thus logged in all log files for each vehicle and shoreside, thus allowing for confirmation that the log files were all part of the same mission.

3 Configuration Parameters of pMissionHash

The following parameters are defined for pMissionHash. If a configuration block is not provided for pMissionHash in the mission file, this is acceptable and no warning will be posted.

Listing 3.1: Configuration parameters for pMissionHash*.*

mission_hash_var:	The variable to which the mission hash is posted. The default is MISSION_HASH.
mhash_short_var:	The variable to which the short version of the mission hash is posted. If this parameter is not specified, this information will not be published. This cannot be the same variable as the mission_hash_var.

An Example MOOS Configuration Block

Listing 2 shows an example MOOS configuration block produced from the following command line invocation:

```
$ pMissionHash --example or -e
```

Listing 3.2: Example configuration of the pMissionHash *application.*

```
1  ============================================================
2  pMissionHash Example MOOS Configuration
3  ============================================================
```

```
 4
 5  ProcessConfig = pMissionHash
 6  {
 7    AppTick   = 4
 8    CommsTick = 4
 9
10    mission_hash_var = MISSION_HASH  // default
11    mhash_short_var  = MHASH_SHORT   // default is disabled
12
13    // Note: If a config block is NOT provided in the mission
14    //       file, NO config warning will be generated.
15  }
```

3.1 Variables Published

The primary output of pMissionHash to the MOOSDB is the mission hash message, or the short mission hash publication if this is enabled.

- APPCAST: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility. Section 5.
- MISSION_HASH: The full mission hash. This may be published to another variable name, depending on the user configuration.
- MHASH_SHORT: The short mission hash. This will only be published if enabled through configuration.

For examples of a full and short mission hash, see Section 5.

Both variables are published immediately upon startup, and once every 30 seconds thereafter.

3.2 Variables Subscriptions

The pMissionHash application subscribes to the following MOOS variables:

- APPCAST_REQ: A request to generate and post a new apppcast report, with reporting criteria, and expiration.
- DB_CLIENTS: The list of running MOOS apps is monitored, checking for pMarineViewer, and posting a warning if present. Both apps should not be running and generating conflicting mission hash values.
- RESET_MHASH: A request to reset the mission hash.

4 Command Line Usage of pMissionHash

The pMissionHash application is typically launched with pAntler, along with a group of other vehicle modules. However, it may be launched separately from the command line. The command line options may be shown by typing:

```
$ uFldMessageHandler --help or -h
```

Listing 4.3: Command line usage for the uFldMessageHandler *tool.*

```
1  ============================================================
2  Usage: pMissionHash file.moos [OPTIONS]
3  ============================================================
4
5  SYNOPSIS:
6  ------------------------------------
7    The pMissionHash app is used for generating a MISSION_HASH
8    posting. Normally this is produced by pMarineViewer. In
9    headless mission not running pMarineViewer, this app can be
10   used instead. They should not be both run unless the mission
11   feature is configured to be disabled in pMarineViewer.
12
13 Options:
14   --alias=<ProcessName>
15       Launch pMissionHash with the given process name
16       rather than pMissionHash.
17   --example, -e
18       Display example MOOS configuration block.
19   --help, -h
20       Display this help message.
21   --interface, -i
22       Display MOOS publications and subscriptions.
23   --version,-v
24       Display the release version of pMissionHash.
25   --web,-w
26       Open browser to:
27       https://oceanai.mit.edu/ivpman/apps/pMissionHash
28
29 Note: If argv[2] does not otherwise match a known option,
30       then it will be interpreted as a run alias. This is
31       to support pAntler launching conventions.
```

5 Terminal and AppCast Output

The pMissionHash application produces some useful information to the terminal and identical content through appcasting. An example is shown in Listing 4 below. On line 2, the name of the local community or vehicle name is listed on the left. On the right, "0/0(95) indicates there are no configuration or run warnings, and the current iteration of pMissionHash is 95.

On line 4, the full mission hash is shown and the variable it is posted to, by default MISSION_HASH. On line 6, the short version of the the mission hash is shown, presumably do to the configuration of mhash_short=MHASH.

Listing 5.4: Example appcast and terminal output of pMissionHash.

```
1  ====================================================================
2  pMissionHash shoreside                                      0/0(95)
3  ====================================================================
4  MISSION_HASH=mhash=250127-1826S-LUSH-ARMY,utc=1738020397.43
5  MHASH=LUSH-ARMY
```

pMissionEval: Evaluating User-Defined Mission Success

January 2025

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
`project-pavlab/appdocs/app_pmissioneval`

1 Overview

The `pMissionEval` application is a tool primarily used in a mission that is part of an automated test. An automated test is typically run headlessly, i.e., no GUI, no human interaction. It is meant to at times complement other MOOS apps that serve to evaluate a mission. For example, in an obstacle avoidance mission, there may be other another app that detects collisions, and another app that randomizes aspects of the obstacle placement. The role of `pMissionEval` is to work in conjunction with the apps of the mission to (a) define when a mission has completed, (b) define what constitutes the result to be reported, and (c) write the mission salient mission configuration aspects and results to a file in a user defined format.

The typical topology is shown below, although it is possible to run pMissionEval in a vehicle commmunity, or in a simple Alpha-like mission with a single MOOS community.

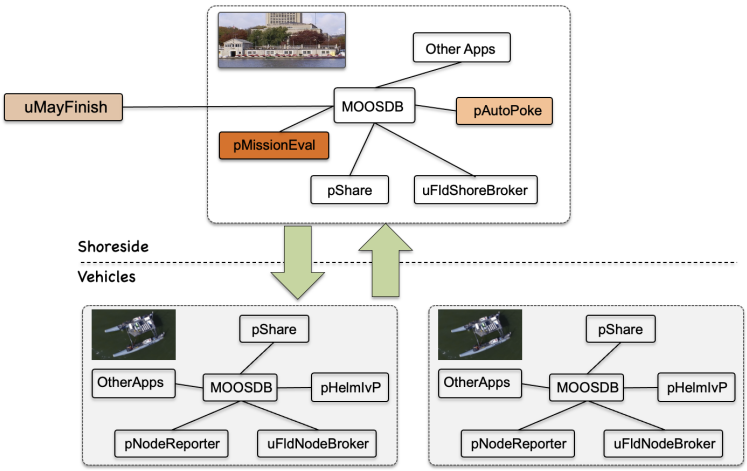

Figure 1: **Typical pMissionEval Topology:** Typically run in the shoreside community, pMissionEval, performs a series of evaluations at events configured by the user, with results posted either to the MOOSDB or to a file, or both.

Note in the figure that two other apps are typically operated in coordination:

- pAutoPoke: When the mission is launched, this app will poke the MOODB with the commands normally provided by a human command to kick off a mission, e.g., `DEPLOY=true`.
- uMayFinish: After the mission is launched, from within a script, this app is launched from with in the script, essentially blocking the script until the mission is finished. This app connect to the MOOSDB and monitor for a posting such as `MISSION_EVALUATED=true`. Then disconnect from the MOOSDB and exit, presumably to allow the remainder of the blocked script to complete bring down the mission.

The primary action produced by pMissionEval is the logging of a single line of text in a named file, e.g., `results.txt`. This file is named in a configuration parameter for pMissionEval. After a mission has been completed, there may be a line produced like:

```
form=alpha, mmod=cycles, grade=pass, mhash=K-POSH-TIME, date=250126, time=1508
```

When pMissionEval determines that evaluation has completed, it will (a) write a result like like the one above to the results file, and (b) post something like MISSION_EVALUATED=true, to match the configuration of uMayFinish to trigger the shut down of the simulation.

2 Four General Mission Types

Consider the following mision types:

- A simple pass/fail sanity check on certain defined features of an app or a behavior. Sometimes referred to as a unit test.

- A performance pass/fail test that requires a lengthy mission duration, with randomization introduced during the mission. Internally the pass/fail determination may be derived from a non Boolean score, e.g., the number of vehicle near-misses over 100 obstacle encounters, but the mission is still a pass/fail test, with the test designer deciding whether success is say 0 near misses or some tolerable number.
- A test that produces a performance score, as in the example above, also possibly involving randomized starting conditions, e.g., obstacle locations. The difference is that this test also has several configuration variations, such as vehicle speed, turning capability, sensor distance, communications drop-out. Rather than this mission being a single component of a CI pipeline, it is part of set of sensitivity test, find the performance of algorithms relative to configuration and environment assumptions. A user may later wish to identify a few of the variations later to be part of a CI pipeline.
- A competition mission that pits two teams, producing a resulting winner and score.

Each of these four mission types are fleshed about a bit more in the following examples.

2.1 The Alpha Mission Pass/Fail Verifying Event Flags

The Alpha mission is the first hello-world mission to where new users of MOOS-IvP are directed on an initial download. A vehicle starts and proceeds through five waypoints in a home-plate pattern. It may traverse the patterntwice before finishing and returning home.

The waypoint behavior has certain event flags defined. The `endflag` is defined for all behaviors and is published when the behavior "completes". For the waypoint behavior, completion is when it has traversed all waypoints. If the waypoint specific parameter `repeat` is set to 1, then the behavior will not complete until it has traverse all five points twice.

The waypoint behavior has two additional event flags unique to this behavior: the `cycle_flag` and `wpt_flag`. The former is published each time the waypoint behavior completes a set of waypoints, and the latter is published each time the behavior arrives at any waypoint. So, for a waypoint behavior configured with five waypoints and `repeat=1`, the `cycle_flag` should be posted twice, and the `wpt_flag` should be posted 10 times.

For such a mission, something like the below result line would be expected.

```
form=alpha, mmod=eflags, grade=pass, mhash=J-BARE-LEAF, date=250126, time=1601
```

The `form=alpha` indicates this is a result of the alpha mission. The `mmod=eflags` is a "mission modification" tag. This is discussed further in Section **??**. A mission may have multiple mods, each to test something slightly different, thus allowing one mission folder to serve multiple purposes.

2.2 The ObAvoid Mission Pass/Fail Verifying Obstacle Avoidance

In the ObAvoid mission, a vehicle is driven around a race-track like pattern, through a rectangular region situated between the two ends of the racetrack pattern. The region is populated with a randomly generated obstacle field. A shoreside MOOS app, `uFldCollObDetect`, runs and montitors the position of vehicles relative to the obstacles in the field. For each obstacle encounter, the closest

point of the encounter is noted, and evaluated. An encounter with an obstacle, with a closest point of approach (CPA), wwithin a certain distance, is considered a collision. A near-miss might be defined as an encounter with a CPA distance within a slightly larger distance.

The automated version of this mission can be configured to run the vehicle for a given number of times around the racetrack, or for a given amount of obstacle encounters. After a certain amount of runs, the mission receives a passing grade if no collisions are detected.

For such a mission, something like the below result line would be expected.

```
form=obavoid, mmod=ob10, grade=pass, mhash=K-ICED-DINO, date=250126, time=1614
```

The `form=obavoid` indicates this is a result from the obavoid mission. The `mmod=ob10` is a modifier that may indicate in this mission there were ten randomly generated obstacles.

2.3 The ObAvoid Mission with Sensitivity Analysis

In the same ObAvoid mission as above, the mission could be graded as a `fail` if a collision with an obstacle is detected. I could also be given a score, from 0 to 100, based on the percentage of near-misses noted for each encounter with an obstacle.

The important distinction is that mission can be varied in several ways. (a) the density of the obstacle field, (b) the minimal guaranteed separation distance between randomly generate obstacles in the field, (c) the range at which the vehicle can detect the obstacle, (d) the turn rate of the vehicle.

An experiment would involve potentially large numbers automated simulations. The result of each simulation would be a single pass/fail grade or score. The goal is plot the relationship between the above mentioned conditions, versus performance.

For such a mission, something like the below result lines would be expected.

```
form=obavoid, mmod=ob6, hits=0, nmisses=0, mhash=A-WEAK-BEER, date=250126, time=1614
form=obavoid, mmod=ob6, hits=0, nmisses=0, mhash=K-LAZY-WEST, date=250126, time=1615
form=obavoid, mmod=ob6, hits=0, nmisses=0, mhash=P-DEAD-ENZO, date=250126, time=1617
...
form=obavoid, mmod=ob10, hits=0, nmisses=2, mhash=M-INKY-SIZE, date=250126, time=1623
form=obavoid, mmod=ob10, hits=0, nmisses=0, mhash=D-COOL-REED, date=250126, time=1624
form=obavoid, mmod=ob10, hits=0, nmisses=1, mhash=M-EASY-VICE, date=250126, time=1637
...
form=obavoid, mmod=ob14, hits=0, nmisses=5, mhash=D-DANK-PHIL, date=250126, time=1639
form=obavoid, mmod=ob14, hits=1, nmisses=8, mhash=G-COLD-KERR, date=250126, time=1641
form=obavoid, mmod=ob14, hits=1, nmisses=2, mhash=C-MAIN-KNOX, date=250126, time=1644
```

The `form=obavoid` indicates this is a result from the obavoid mission. The `mmod=ob10` is a modifier that may indicate in this mission there were ten randomly generated obstacles. The overall test involves several instances of each configuration. In this example, the number of obstacles presumably increases the difficulty in obstacle avoidance and the data should enable a plot relating obstacle density with to the frequency of hits or near misses.

2.4 The Rescue Mission Head-to-Head Competion

The Rescue mission is a head-to-head competition mission from the MIT 2.680 Marine Autonomy class. This mission is essentially in the style of an Easter egg hunt. Two vehicles are deployed to rescue N swimmers, by visiting each swimmer location. The vehicle to visit the majority of the N swimmers wins. There is an game manager app running in the shoreside community called uFldRescueMgr. This app will publish something like

```
WINNER   = blue
SCORE    = blue=9 # green=7
SWIMMERS = 17

COMPETITION_COMPLETE = true
```

When launched with a GUI, the participants could watch the mission play out, and check the scoreboard in the appcasting output of uFldRescueMgr to see the winner and indication of completeness. In an automated headless mode, pMissionEval would wait for COMPETITION_COMPLETED to be true, and publish a line to the results file looking something like:

```
form=rescue, winner=blue, score=blue=9 # green=7, mhash=K-PLUS-MOTH, date=250126, time=1644
```

Since the competition involves randomly generated starting positions and locations of swimmers, an experiment may involve many such automated competition missions to get a good feel for the team with the better strategy.

3 Basic Operation

In any mission, pMissionEval will do essentially the following three things:

- Wait until the proper point in the mission
- Evaluate the results of the mission
- Post the results to either a file or to the MOOSDB, or both

In an automated, headless mission, there is always the concern about having an absolute timeout, in case for some reason the first stage above fails to happen. However, pMissionEval is typically not concerned about this. The uMayFinish app is normally configured with an absolute for aborting a mission.

3.1 Conditions and Mission Evaluation

pMissionEval can be configured with one or more logic conditions confiured to convey a mission stage or mission completeness. A simple example is

```
lead_condition = ARRIVED = true
```

When configured this way, pMissionEval will register for the MOOS variable ARRIVED. When it receives mail with a value of `true`, the mission will be evaluated and results posted. Evaluation in terms of pass/fail is determined by an additional two kinds of conditions, *pass conditions* and *fail conditions*. A result of *pass* will be award if (a) all pass conditions hold and (b) no fail condition holds. For example, the above line may be further configured with:

```
pass_condition = ODOMETRY < 1500
pass_condition = FOUND_OBJECT = true
fail_condition = COLLISIONS > 0
```

The above three lines will be evaluated only when the lead condition is satisfied, ARRIVED=true. If both pass conditions are satisfied, and none of the fail conditions are satisfied, then the mission is (a) considered to be evalutated, and (b) the pass/fail result is set to pass.

3.2 Results of a Mission Evaluation

When a mission is considered to be evaluated (lead conditions satisfied), either or both of the above actions are taken:

- User configured result flags are posted to the MOOSDB
- User configures result lines are written to a file.

Result flags come in one of three flavors: *pass* flag, *fail* flags, and simply *result* flags. The first is published only when the pass/fail result passes, the second only when it fails, and the result flag is published either way. Here are some examples:

```
pass_flag = RESULT=success
fail_flag = RESULT=failed
result_flag = MISSION_EVALUATED=true
```

The other action taken on evaluation is the publication of a result line to a result file. For example, if configured:

```
result_file = result.txt
result_col = result=$[GRADE]
```

The action on mission evaluation will be to write a single line to the file `results.txt`:

```
result=pass
```

Continuing with the above example, we know that if the result passes then the two conditions involving ODOMETRY and FOUND_OBJECT must have passed. But way may want to also know the actual odometry value. Using another column

```
result_col = odometry=$[ODOMETRY]
```

then the line written to the results file would be:

```
result=pass, odometry=1451
```

Any number of result file columns can be configured.

3.3 Multi-Part Tests

A multi-part test may be used for evaluating a mission at different stages. In a single mission, perhaps a vehicle is designed to transit to a point, then loiter for some period of time, and then return home, finally setting `ARRIVED=true`. Rather than just checking to see if that post was eventually made, the intermediate stages can be verified too with a *multi-aspect logic sequence*. Each aspect is comprised of one or more lead conditions and one or more pass/fail conditions.

```
lead_condition = ARRIVED_MID = true
pass_condition = ODOMETRY < 500

lead_condition = LOITER_FINISH = true
pass_condition = ODOMETRY < 1000

lead_condition = ARRIVED = true
pass_condition = ODOMETRY < 1500
```

Each pair above is *aspect* and the group is called a *logic sequence*. The overall mission will not be considered evaluated and results will not be posted or written to a file, until the full sequence is evaluated. There are no limits to the number of lead, pass and fail conditions per aspect, and there are no limits on the number of aspects. The appcasting output of `pMissionEval` will show the current progress of a multi-aspect logic sequence.

Note: Unlike configuration parameters in nearly all other MOOS apps, the *order* of the configuration lines is significant.

3.4 Structuring the Results File

The results file may have as many columns as the user desires, and there are no minimums or limits on the columns. Columns are by default separated by white space, but they can be configured to be separated by a comma (csp, for comma-separated-pairs) with:

```
report_line_format = csp
```

The results file will likely contain results from multiple missions, or mission types. Some consideration should be made to add information to report lines with an eye toward how the results file will be later processed, as part of a CI pipeline or a file for generating plotted data. With this in mind, there are a number of provisions for macros available for configuring report lines, described next.

3.5 Report Macros

Any of the flags (result, pass, or fail flags) may include a *macro*. Likewise, any report column may also contain a macro. Macros are of the form `$[MACRO]`. A macro may be something built into the app, such as `$[DATE]`, or others listed below. If a `$[MACRO]` is not one of the built-in macros, the pMissionEval will interpret this to mean that the macro refers to a MOOS variable.

For example, consider a mission with an odometry app running and continuously, posting to the variable ODOMETRY. We may want to construct an evaluation that uses either (a) a result flag, or (b) a report column, that uses the odometry information:

```
result_flag = DIST = $[ODOMETRY]
```

or:

```
report_column = dist=$[ODOMETRY]
```

Built-in macros:

- GRADE: This is the mission result value. It will be either "pending", "pass", or "fail".
- MHASH_SHORT: The short version of a mission hash string. An example long version would be `"mhash=250117-1614B-MEEK-EROS,utc=1737148477.26"`. The short version would simply be `"MEEK-EROS"`.
- MHASH_LSHORT: A longer version of the short version, keeping the single letter preceding it. For example, `"B-MEEK-EROS"`, instead of `"MEEK-EROS"`.
- MHASH: The full version of a mission hash, e.g., `"mhash=250117-1614B-MEEK-EROS,utc=1737148477.26"`.
- MISSION_HASH: Same as `$[MHASH]`.
- MHASH_UTC: The timestamp component of a mission hash string. An example long version would be `"mhash=250117-1614B-MEEK-EROS,utc=1737148477.26"`. The timestamp component would simply be `"1737148477.26"`.
- MISSION_FORM: This is the value set in the mission_form configuration parameter. It should be set to the colloquial name of the mission, e.g., the "Alpha" mission.
- MISSION_MOD: This is the value set in the mission_mod configuration parameter. If a mission has several mods, then this is the place to distinguish that.
- WALL_TIME: The wall time is the DB_UPTIME value, divided by the time warp value.

Built-in Time and Date macros:

- `DATE`: A string in the format of `YYYY:MM:DD`.
- `TIME`: A string in the format of `HH:MM:SS`.
- `YEAR`: A string in the format of `YYYY`.
- `MONTH`: A string in the format of `MM`.
- `DAY`: A string in the format of `DD`.
- `HOUR`: A string in the format of `HH`.
- `MIN`: A string in the format of `MM`.
- `SEC`: A string in the format of `SS`.

4 Four Example Missions

4.1 The Alpha Mission Pass/Fail Verifying Event Flags

Returning to the Alpha mission Section 2.1. The goal is to verify the end flag, waypoint flag and cycle flag features. First, the waypoint behavior is configured to increment a counter on each posting of the flags as fallows.

```
wptflag   = WPT_FLAG=$[CTR1]
cycleflag = CYCLE_FLAG=$[CTR2]
endflag   = ARRIVED=true
```

Two tests are made in two logic aspects:

```
lead_condition = CYCLE_FLAG = 1
pass_condition = WPT_FLAG = 5

lead_condition = ARRIVED = true
pass_condition = WPT_FLAG = 10
pass_condition = CYCLE_FLAG = 2

mission_form  = alpha_ufld
mission_mod   = mod1
report_file   = results.txt
report_column = grade=$[GRADE]
report_column = form=$[MISSION_FORM]
report_column = mmod=$[MMOD]
report_column = mhash_short=$[MHASH_SHORT]
report_column = lshort_short=$[MHASH_LSHORT]

result_flag = MISSION_EVALUATED = true
```

4.2 The ObAvoid Mission Pass/Fail Verifying Obstacle Avoidance

Returning to the obstacle avoidance mission described in Section 2.2. This is misison `02-obavoid` in the `missions-auto` repository. A vehicle is driven through an obstacle field with a dedicated MOOS app monitoring each obstacle encounter. This app is called `uFldCollObDetect`. It publishes three key variables:

- `OB_TOTAL_ENCOUNTERS`: Incremented each time a vehicle passes an obstacle.
- `OB_TOTAL_COLLISIONS`: Incremented each time a vehicle collides with an obstacle.
- `OB_TOTAL_NEAR_MISSES`: Incremented each time a vehicle encounter has a near miss with an obstacle.

For each of the above, the app is configured with range threshold, e.g, a 20 meter CPA to an obstacle counts as an encounter, a 5 meter CPA is a near miss, and 1 meter CPA is considered a collision.

A simple pass/fail test can be constructed with the below `pMissionEval` configuration:

```
lead_condition = (OB_TOTAL_ENCOUNTERS > $(TEST_ENCOUNTERS))

pass_flag   = OB_TOTAL_COLLISIONS = 0
result_flag = MISSION_EVALUATED = true

report_column = min_sep=$(SEP)
report_column = grade=$[GRADE]
report_column = encounters=$[OB_TOTAL_ENCOUNTERS]
report_file = results.log
```

This test can be conducted by running, using time warp 10:

```
$ xlaunch.sh 10
```

This should produce a line a output in file `results.log` similar to:

```
min_sep=10  grade=pass  encounters=25
```

Note in this mission, the launch script accepts command line arguments setting the (a) the minimum separation between obstacles, and (b) the total number of encounters tested. The defaults are 10 and 25 respectively as shown in the above result line. The values are expanded in the `nsplug` macros `$(SEP)` and `$(TEST_ENCOUNTERS)`. The mission could be launched instead with:

```
$ xlaunch.sh 10 --sep=12 --enc=40
```

This should produce a line a output in file `results.log` similar to:

```
min_sep=12  grade=pass  encounters=40
```

4.3 The ObAvoid Mission with Sensitivity Analysis

Staying with the same obstacle avoidance mission, a sequence of tests is constructed. The end result will be a `results.log` file with many lines, each representing the results of distict tests, with perhaps different configuration parameters. For this reason, each line in the results file is configured with more information:

```
lead_condition = (OB_TOTAL_ENCOUNTERS > $(TEST_ENCOUNTERS))

pass_flag   = OB_TOTAL_COLLISIONS = 0
result_flag = MISSION_EVALUATED = true

report_column = form=obavoid
report_column = min_sep=$(SEP)
report_column = grade=$[GRADE]
report_column = collisions=$[OB_TOTAL_COLLISIONS]
report_column = near_misses=$[OB_TOTAL_NEAR_MISSES]
report_column = encounters=$[OB_TOTAL_ENCOUNTERS]
report_file = results.log
```

Now a set of tests can be launched, each varying the minimum separation distance between obstacles in the randomly generated obstacle field:

```
$ xlaunch.sh --sep=14 10
$ xlaunch.sh --sep=12 10
$ xlaunch.sh --sep=10 10
$ xlaunch.sh --sep=8 10
$ xlaunch.sh --sep=6 10
$ xlaunch.sh --sep=4 10
```

This should produce several lines of output in the file `results.log` similar to:

```
form=obavoid  min_sep=14  grade=pass collisions=0  near_misses=0  encounters=25
form=obavoid  min_sep=12  grade=pass collisions=0  near_misses=0  encounters=25
form=obavoid  min_sep=10  grade=pass collisions=0  near_misses=1  encounters=25
form=obavoid  min_sep=8  grade=pass collisions=0  near_misses=0  encounters=25
form=obavoid  min_sep=6  grade=pass collisions=0  near_misses=2  encounters=25
form=obavoid  min_sep=4  grade=pass collisions=0  near_misses=4  encounters=25
```

5 Configuration Parameters for pMissionEval

The following parameters are defined for pMissionEval. A more detailed description is provided in other parts of this section. Parameters having default values are indicated.

Listing 5.1: Configuration Parameters for pMissionEval.

fail_condition: A logic condition that must *not* be satisfied or otherwise the overall mission will be considered successful. Section 3.1.

fail_flag: A flag to be publish once the result is known and is mission has been deemed to be not successful. Section 3.2.

lead_condition: A logic condition that must be satisfied before the mission can be evaluated. Section 3.1.

pass_condition: A logic condition that must be satisfied for the overall mission to be considered successful. Section 3.1.

pass_flag:	A flag to be publish once the result is known and is mission has been deemed to be successful. Section 3.2.
mission_form:	Set the name of the mission, e.g., "alpha", or "berta" etc., so the MISSION_FORM macro can be expanded in any reporting.
report_column:	Define a column of output in the in the report file. See Section 3.4.
report_file:	Name the report file to receive output of results. See Section 3.4.
report_line_format:	Determines the separator between report columns. By default it is white space, "white", but can also be set to us a comma, "csp".
result_flag:	A flag to be publish once the result is known. Section 3.2.

6 Publications and Subscriptions of pMissionEval

The interface for pMissionEval, in terms of publications and subscriptions, is described below. This same information may also be obtained from the terminal with:

```
$ pMissionEval --interface or -i
```

6.1 Variables Published by pMissionEval

The publications of pMissionEval are mostly configurable in the user specified pass, fail and result flags. There are a few hard-coded publications:

- APPCAST: Contains an appcast report identical to the terminal output. Appcasts are posted only after an appcast request is received from an appcast viewing utility. Section 7.
- MISSION_LCHECK_STAT: See Section **??**.
- MISSION_RESULT: See Section **??**.
- MISSION_EVALUATED: See Section **??**.

6.2 Variables Subscribed for by pMissionEval

The subscriptions for pMissionEval mostly are dictated by the user-specified test conditions, i.e., the lead_condition, pass_condition and fail_condition parameters. Users may also specified any MOOS variable to be echoed in any user specified event flags, and in publications to a results file. There are a few hard-coded subscriptions:

- APPCAST_REQ: A request to generate and post a new appcast report, with reporting criteria, and expiration.
- DB_UPTIME: The amount of seconds that the local MOOSDB has been running.
- MISSION_HASH: The mission hash posting contains a few fields that need to be know to support a couple mission hash macros. See Section 3.5.
- APPNAME_PID: "ignore" ("none"). Section **??**.

7 Terminal and AppCast Output

The pMissionEval application produces some useful information to the terminal on every iteration of the application. An example is shown in Listing 2 below. This application is also appcast enabled, meaning its reports are published to the MOOSDB and viewable from any uMAC application or pMarineViewer. The counter on the end of line 2 is incremented on each iteration of pMissionEval, and serves a bit as a heartbeat indicator. The "0/0" also on line 2 indicates there are no configuration or run warnings detected.

The output in the below example comes from the example described in Section **??**.

Listing 7.2: Example terminal or appcast output for pMissionEval*.*

```
1  ================================================================
2  pMissionEval shoreside                                   0/0(25)
3  ================================================================
4  Overall State: pending
5  ============================================
6   logic_tests_stat: unmet_lead_cond: [ARRIVED=true]
7       result flags: 1
8         pass flags: 0
9         fail flags: 0
10        curr_index: 0
11       report_file: results.log
12
13 ============================================
14 Supported Macros:
15 ============================================
16
17 Macro         CurrVal
18 ------------  -------------
19 ENCOUNTER     0
20 MHASH_SHORT   GOLD-BABY
21 MHASH_UTC     1737549326.28
22 MISSION_FORM  01-colavd
23 WALL_TIME     0
24
25 ============================================
26 Logic Sequence:
27 ============================================
28
29 Index  Status   Cond  PFlag  FFLag
30 -----  -------  ----  -----  -----
31 0      current  1     0      0
32
33 ============================================
34 Reporting:
35 ============================================
36   report_file: results.log
37    report_col: m_form=$[MISSION_FORM]
38    report_col: 5
39    report_col: $[ENCOUNTER]
40    report_col: mhash=$[MHASH_SHORT]
41    report_col: utc=$[MHASH_UTC]
42    report_col: wall=$[WALL_TIME]
```

```
43    latest_line:
44
45  ================================================================
46  Most Recent Events (4):
47  ================================================================
48  [4.80]: Mail:DB_UPTIME
49  [2.78]: Mail:DB_UPTIME
50  [2.27]: Mail:MISSION_HASH
51  [0.76]: Mail:DB_UPTIME
```

The first few lines (4-11) provide a high-level snapshot of the current state. Line 4 shows the overall state of the test. Lines 13-23 show all macros available in posting flags or generating column output in a report file. Lines 25-31 show the prevailing logic sequence. A sequence may have several aspects (one per line in this table). Lines 33-43 show the configuration of the report file output. Line 43 shows the latest line written to this file. Line 45 is the start of the most recent events. For pMissionEval, for now, the events are just the received mail registered for this app.

All flags for all behaviors also have certain macros defined for event postings. Two such macros are `$[CTR1]` and `$[CTR2]`. They are counters that are incremented each time they are used in a posting. So if the cyle_flag and wpt_flag parameters are set in the waypoint behavior of the Alpha mission:

```
wpt_flag = WFLAG=$[CTR1]
cycle_flag = CFLAG=$[CTR1]
repeat = 1
end_flag = ARRIVED=true
```

The vehicle will traverse the five points twice, and the MOOS variables WFLAG and CFLAG should have the values 10 and 2 respectively, and ARRIVED should be set to `true`.

- It encapsulates mission evaluation to be defined by the pMissionEval configuration block. This allows automated scripts to treat a mission generically: launch it, monitor for its ending, and shut it down.
- It can facilitate multiple component evaluations in a single mission, distilling it down to a single pass/fail grade.
- It allows for custom configuration of a report file that facilitates easier post-processing for plotting result data.
-

7.1 Relation to uMayFinish

In an automated mission, some entity is monitoring one or more of the MOOS communities involved in the situation, typically the shoreside, for an event that indicates that the mission is complete and evaluations are completed and registered somewhere. After this point, the mission (all MOOS apps) are automatically shut down.

The uMayFinish application ma